FACULTÉ DES SCIENCES DE SFAX	*Département d'informatique et des Communications* Année Universitaire : 2026-2027
	TP3 REST : Microservices « Calculatrice » avec Spring Boot **Matière : Architecture MicroServices** **Filière : MPSRCC1** **Enseignantes : Corinne Amel ZAYANI, Amal ABID**

Présentation

Dans le TP2, vous avez développé un service web REST « Calculatrice » avec JAX-RS, construit sous forme de fichier .war et déployé dans un serveur Tomcat installé à part. Dans ce TP, vous réalisez la même calculatrice, mais sous forme de **microservices** avec **Spring Boot** : deux petites applications indépendantes, `calcul-service` et `historique-service`, qui démarrent chacune avec leur propre serveur embarqué et qui communiquent entre elles par HTTP.

Les fonctionnalités sont volontairement identiques à celles du TP2 (mêmes URI, mêmes verbes, mêmes codes de statut). Vous pourrez ainsi vous concentrer sur ce qui change : l'outil (Spring Boot au lieu de JAX-RS et Tomcat) et l'architecture (plusieurs microservices au lieu d'un seul service web).

Objectifs

À la fin de ce TP, vous saurez :

- Expliquer la différence entre un service web déployé dans un serveur et un microservice autonome.
- Créer un projet Spring Boot avec Spring Initializr.
- Écrire un contrôleur REST avec les annotations de Spring (`@RestController`, `@GetMapping`, `@RequestParam`, `@PathVariable`, `@RequestBody`…).
- Lancer un microservice sous forme de fichier .jar exécutable, sur un port choisi.
- Faire communiquer deux microservices par HTTP avec `RestClient`.
- Mettre en correspondance les annotations JAX-RS du TP2 et celles de Spring.

Prérequis

Vérifiez que les outils suivants sont installés sur votre poste.

- JDK 17+ (JDK 21 recommandé)
- Maven 3.9+ (intégré à IntelliJ)
- IntelliJ IDEA
- Postman (toute version)

- Curl (toute version)

Tomcat n'est plus nécessaire. Spring Boot embarque son propre serveur Tomcat à l'intérieur de l'application : c'est l'une des différences principales avec le TP2.

Version de Spring Boot : ce TP a été rédigé avec Spring Boot 4.x, qui exige un JDK 17 ou plus récent. Si Spring Initializr vous propose une version 3.5.x, la dépendance web s'appelle `spring-boot-starter-web` au lieu de `spring-boot-starter-webmvc` , tout le reste du TP est identique. Pour assurer la compatibilité avec Spring Cloud, utilisé au TP4, ce TP impose la version 4.0.9 (étape B.1).

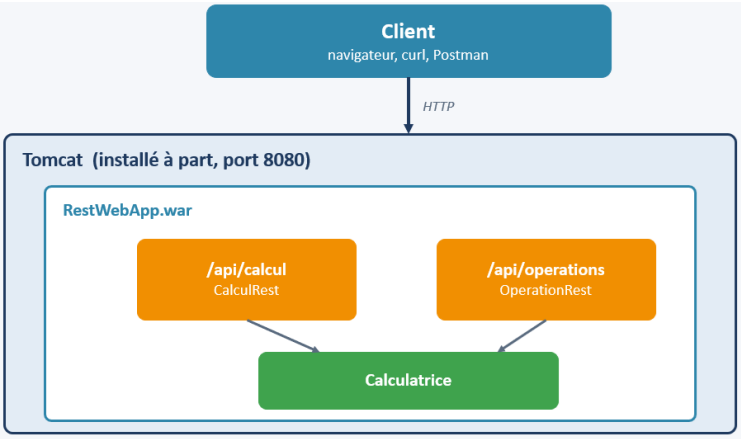
Ports : les deux microservices écoutent sur les ports 8081 et 8082. Si l'un d'eux est déjà occupé sur votre poste, arrêtez le programme concerné ou choisissez un autre port à l'étape. Le Tomcat du TP2, sur le port 8080, ne gêne pas.

Partie A : Du service web au microservice

Avant de coder, on compare les deux approches. Cette partie se fait sans ordinateur.

A.1 Rappel : l'application du TP2

Dans le TP2, une seule application web `RestWebApp.war` contenait tout : la logique de calcul (classe `Calculatrice`), la ressource `/calcul` et la ressource `/operations`. Elle était déployée dans un serveur Tomcat installé séparément, et accessible à l'adresse `http://localhost:8080/RestWebApp/api/...`.



Architecture du TP2 : un service web dans un serveur

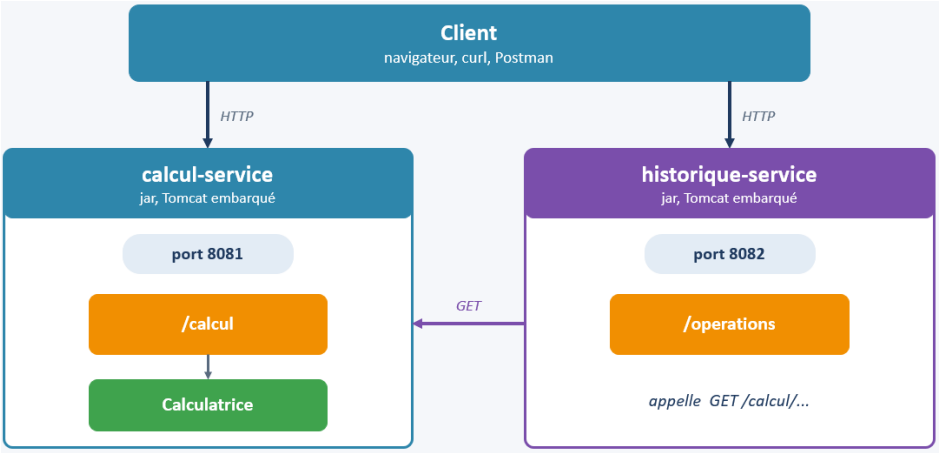
A.2 L'architecture visée dans ce TP

Un **microservice** est une petite application autonome, responsable d'une seule fonctionnalité métier, qui démarre seule (avec son propre serveur embarqué), qui écoute sur son propre port et qui communique avec les autres microservices par HTTP. Chaque microservice peut être développé, démarré, arrêté et redéployé indépendamment des autres.

La calculatrice est découpée en deux microservices :

- `calcul-service` (port 8081) : effectue les quatre calculs. Il expose la ressource `/calcul`.

- `historique-service` (port 8082) : enregistre les opérations. Il expose la ressource `/operations`. Pour calculer le résultat d'une opération, il **appelle** `calcul-service` par HTTP : il ne contient aucune logique de calcul.



Architecture du TP3 : deux microservices qui communiquent par HTTP

Les URI sont les mêmes qu'au TP2, à deux différences près : il n'y a plus de nom d'application ni de préfixe `/api` dans l'adresse, et chaque famille d'URI est servie par un microservice différent, donc sur un port différent.

Verbe	URI (microservice)	Rôle
GET	`:8081/calcul/somme?a=2&b=3`	Retourne a + b
GET	`:8081/calcul/soustraction?a=2&b=3`	Retourne a − b
GET	`:8081/calcul/produit?a=2&b=3`	Retourne a × b
GET	`:8081/calcul/division?a=6&b=3`	Retourne a ÷ b
POST	`:8082/operations`	Crée une opération (le calcul est demandé à calcul-service, puis enregistré)
GET	`:8082/operations`	Retourne toutes les opérations enregistrées
GET	`:8082/operations/{id}`	Retourne l'opération numéro id
PUT	`:8082/operations/{id}`	Modifie l'opération numéro id
DELETE	`:8082/operations/{id}`	Supprime l'opération numéro id

A.3 Service web (TP2) et microservice (TP3) : ce qui change

Le tableau suivant résume les différences. Vous le retrouverez, point par point, dans la suite du TP.

Aspect	Service web du TP2	Microservice du TP3
Technologie	JAX-RS (implémentation Jersey)	Spring Boot (Spring MVC)
Découpage	Une seule application : calcul et historique dans le même .war	Deux applications indépendantes : `calcul-service` et `historique-service`
Résultat de la construction	Fichier .war (archive web)	Fichier .jar exécutable, qui contient l'application **et** le serveur
Serveur	Tomcat installé à part, on y dépose le .war	Tomcat embarqué dans le .jar ; l'application démarre seule (`java -jar`)
Port	8080, celui de Tomcat, partagé par toutes les applications déployées	Un port par microservice (8081, 8082), choisi dans `application.properties`
Adresse	`http://localhost:8080/RestWebApp/api/calcul/somme`	`http://localhost:8081/calcul/somme`
Activation	Classe `RestApplication` annotée `@ApplicationPath`	Classe `...Application` annotée `@SpringBootApplication`, avec une méthode `main`
Ressource REST	Classe annotée `@Path`	Classe annotée `@RestController` et `@RequestMapping`
Verbes HTTP	`@GET`, `@POST`, `@PUT`, `@DELETE`	`@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`
Paramètres	`@QueryParam`, `@PathParam`, corps avec `@Consumes`	`@RequestParam`, `@PathVariable`, corps avec `@RequestBody`
Réponse HTTP	`Response`	`ResponseEntity`
Conversion JSON	JSON-B (`jersey-media-json-binding`)	Jackson (inclus dans Spring Boot)
Création des objets	`new Calculatrice()` ; attributs `static` pour partager l'historique	Objets gérés par Spring (`@Service`, injection par constructeur, cf. chapitre 1 sur l'IoC/DI) ; un seul exemplaire par défaut
Communication	Appel de méthode Java entre `OperationRest` et `Calculatrice`	Requête HTTP entre `historique-service` et `calcul-service`

A.4 Questions de conception

Répondez par écrit, en une ou deux phrases chacune.

1. Dans le TP2, le résultat d'une opération était calculé par un simple appel de méthode. Dans ce TP, il est obtenu par une requête HTTP vers un autre microservice. Citez un avantage et un inconvénient de ce choix.
2. Si `calcul-service` est arrêté, quelles requêtes du tableau A.2 continuent de fonctionner ? Lesquelles échouent ?
3. Pourquoi les deux microservices ne peuvent-ils pas écouter sur le même port ?
4. Quel code de statut HTTP `historique-service` doit-il renvoyer quand il ne parvient pas à joindre `calcul-service` ? (Indication : ce n'est ni une erreur du client, ni un bug du serveur ; cherchez dans la famille des codes 5xx.)

Partie B : Création du projet calcul-service

Un projet Spring Boot est un projet Maven ordinaire. On le génère avec **Spring Initializr**, un site qui prépare le fichier `pom.xml` et la classe principale.

B.1 Générer le projet avec Spring Initializr

1. Ouvrez le site `https://start.spring.io` dans un navigateur.
2. Remplissez le formulaire avec les valeurs ci-dessous.
    a. Project : Maven
    b. Language : Java
    c. Spring Boot : 4.0.9 (à choisir dans la liste : ce n'est pas la valeur cochée par défaut)
    d. Group : com.exemple
    e. Artifact : calcul-service
    f. Name : calcul-service
    g. Package name : com.exemple.calcul
    h. Packaging : Jar
    i. Java La version de votre JDK (17 ou 21)
3. Dans la zone **Dependencies**, cliquez sur **Add dependencies...**, puis choisissez **Spring Web**.
4. Cliquez sur **Generate** : le navigateur télécharge l'archive `calcul-service.zip`.
5. Décompressez l'archive, puis ouvrez le dossier obtenu dans IntelliJ IDEA par **File > Open**. IntelliJ reconnaît le projet Maven et télécharge les dépendances (patientez jusqu'à la fin de l'indexation).

IntelliJ IDEA Ultimate propose le même formulaire par **File > New > Project > Spring Boot**, le résultat est identique.

B.2 Structure attendue

À la fin du TP, le projet `calcul-service` aura la structure suivante. Les fichiers marqués (généré) existent déjà. Vous créerez les autres, chacun dans le sous-paquetage indiqué de `com.exemple.calcul`.

```
calcul-service/
├── pom.xml                                        (généré)
└── src/
    └── main/
        ├── java/com/exemple/calcul/
        │   ├── CalculServiceApplication.java      (généré : démarrage)
        │   ├── controller/
        │   │   └── CalculController.java          (ressource /calcul)
        │   ├── service/
        │   │   └── Calculatrice.java              (logique métier)
        │   └── model/
        │       └── Resultat.java                  (objet renvoyé en JSON)
        └── resources/
            └── application.properties             (généré)
```

Organisation en paquetages. Les classes sont rangées par rôle, comme dans la plupart des applications Spring. Chaque sous-paquetage se trouve sous `com.exemple.calcul` : Spring les trouve donc automatiquement (voir B.4). Créez-les avant d'écrire les classes (clic droit sur le paquetage > New > Package).

- `controller` : les classes qui reçoivent les requêtes HTTP (@RestController) ;
- `service` : la logique métier (@Service) ;
- `model` : les objets de données échangés en JSON ;
- `client` : l'appel HTTP vers un autre microservice (uniquement dans historique-service, voir F.3).

Il n'y a volontairement pas de paquetage `dao` ou `repository` : les données sont conservées en mémoire, sans base de données. Ce paquetage apparaîtrait si l'on ajoutait une base de données (JPA) ; `Operation` deviendrait alors une entité.

B.3 Lire le fichier pom.xml

Contrairement au TP2, vous n'avez rien à remplacer : Spring Initializr a écrit le `pom.xml`. Ouvrez-le et retrouvez-y les éléments suivants (les dépendances de test ont été omises ici, et le numéro de version peut différer).

```
<project ...>
    <modelVersion>4.0.0</modelVersion>

    <!-- Projet parent : fixe les versions des dépendances Spring -->
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.0.9</version>
        <relativePath/>
    </parent>

    <groupId>com.exemple</groupId>
    <artifactId>calcul-service</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>calcul-service</name>
    <!-- Pas de balise <packaging> : la valeur par défaut est jar -->

    <properties>
        <java.version>17</java.version>
```

```
    </properties>

    <dependencies>
        <!-- Spring MVC (annotations REST), Jackson (JSON), Tomcat embarqué -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <!-- Produit un .jar exécutable (application + serveur) -->
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

Points à retenir :

- `spring-boot-starter-parent` est le projet parent : il choisit, pour vous, des versions compatibles de toutes les bibliothèques. C'est pourquoi la dépendance `spring-boot-starter-webmvc` n'a pas de balise `<version>`.
- Un *starter* est une dépendance qui en regroupe plusieurs. `spring-boot-starter-webmvc` apporte à la fois Spring MVC (l'équivalent de JAX-RS), Jackson (l'équivalent de JSON-B) et le serveur Tomcat embarqué. Au TP2, il fallait quatre dépendances et un Tomcat installé à part.
- Il n'y a pas de balise `<packaging>war</packaging>` : le projet produit un fichier .jar.
- `spring-boot-maven-plugin` rend ce .jar exécutable : il y range l'application, toutes ses dépendances et le serveur Tomcat.

B.4 La classe principale (générée)

Ouvrez la classe `CalculServiceApplication`, créée par Spring Initializr dans le paquetage `com.exemple.calcul`.

```
package com.exemple.calcul;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class CalculServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(CalculServiceApplication.class, args);
    }
}
```

Cette classe remplace à la fois la classe `RestApplication` du TP2 et le serveur Tomcat :

- `@SpringBootApplication` active Spring, la configuration automatique et la recherche des classes annotées (`@RestController`, `@Service`…) dans le paquetage `com.exemple.calcul` et ses sous-paquetages. Une classe placée hors de ce paquetage ne serait pas trouvée.
- La méthode `main` démarre l'application comme un programme Java ordinaire : Spring crée les objets, démarre le Tomcat embarqué et se met à l'écoute.

Partie C : Premier microservice : la somme

Vous écrivez deux classes, vous choisissez le port, vous lancez le microservice, puis vous testez. Toutes les classes sont créées dans le paquetage Toutes les classes sont créées dans les sous-paquetages de `com.exemple.calcul` indiqués en B.2.

C.1 La logique métier : classe Calculatrice

La classe est celle du TP2, à une différence près : l'annotation `@Service`. Elle indique à Spring de créer un objet `Calculatrice` au démarrage et de le fournir aux classes qui en ont besoin. **C'est le mécanisme d'injection de dépendances vu au TP1.** Placez-la dans le paquetage `service`.

```
package com.exemple.calcul.service;

import org.springframework.stereotype.Service;

@Service
public class Calculatrice {

    public int somme(int a, int b) {
        return a + b;
    }

    public int soustraction(int a, int b) {
        return a - b;
    }

    public int produit(int a, int b) {
        return a * b;
    }

    public double division(int a, int b) {
        if (b == 0) {
            throw new ArithmeticException("Division par 0 impossible");
        }
        return (double) a / b;
    }
}
```

C.2 Le contrôleur REST : classe CalculController

Ce que JAX-RS appelle une *ressource*, Spring l'appelle un *contrôleur*. Le rôle est le même : chaque méthode annotée répond à une requête HTTP. Placez-le dans le paquetage `controller`. Comme il utilise `Calculatrice`, il doit l'importer (déjà indiqué dans le code ci-dessous).

```java
package com.exemple.calcul.controller;

import com.exemple.calcul.service.Calculatrice;

import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/calcul")
public class CalculController {

    private final Calculatrice calculService;

    // Injection par constructeur : Spring fournit l'objet Calculatrice
    public CalculController(Calculatrice calculService) {
        this.calculService = calculService;
    }

    @GetMapping(value = "/somme", produces = MediaType.TEXT_PLAIN_VALUE)
    public ResponseEntity<String> somme(@RequestParam("a") int a,
                                        @RequestParam("b") int b) {
        int result = calculService.somme(a, b);
        return ResponseEntity.ok(String.valueOf(result));
    }
}
```

Correspondance avec le TP2 :

Spring (TP3)	JAX-RS (TP2)	Rôle dans cet exemple
`@RestController`	(pas d'équivalent direct)	***A compléter***
`@RequestMapping("/calcul")`	`@Path("/calcul")` sur la classe	***A compléter***
`@GetMapping("/somme")`	`@GET` + `@Path("/somme")`	***A compléter***
`produces = MediaType.TEXT_PLAIN_VALUE`	`@Produces(MediaType.TEXT_PLAIN)`	***A compléter***
`@RequestParam("a") int a`	`@QueryParam("a") int a`	***A compléter***
`ResponseEntity.ok(...)`	`Response.ok(...).build()`	***A compléter***

Remarque. Aucun `new Calculatrice()` : le contrôleur reçoit l'objet par son constructeur. Spring crée un seul objet `Calculatrice` et un seul objet `CalculController`, réutilisés pour toutes les requêtes (alors que JAX-RS créait un nouvel objet `CalculRest` à chaque requête).

C.3 Vérifier l'emplacement des classes

Assurez-vous que `Calculatrice` est dans `com.exemple.calcul.service` et `CalculController` dans `com.exemple.calcul.controller`. Ces paquetages doivent se trouver sous `com.exemple.calcul`, celui de `CalculServiceApplication`. Sinon, Spring ne les trouvera pas et toutes les requêtes répondront 404.

C.4 Choisir le port : application.properties

Par défaut, le Tomcat embarqué écoute sur le port 8080. Comme vous allez lancer deux microservices en même temps, chacun doit avoir son propre port. Ouvrez le fichier `src/main/resources/application.properties` et complétez-le ainsi :

```
spring.application.name=calcul-service
server.port=8081
```

La première ligne, générée par Initializr, donne un nom à l'application. La seconde fixe le port d'écoute.

C.5 Lancer le microservice

Il n'y a rien à déployer : le microservice est un programme Java que l'on exécute. Trois moyens, au choix:

- **Dans IntelliJ IDEA :** ouvrez `CalculServiceApplication` et cliquez sur la flèche verte à côté de la méthode `main` (ou **Run > Run 'CalculServiceApplication'**).
- **Avec Maven, sans construire le .jar :** dans un terminal, à la racine du projet, exécutez `mvn spring-boot:run`.
- **Avec le .jar exécutable :** exécutez `mvn clean package`, puis `java -jar target/calcul-service-0.0.1-SNAPSHOT.jar`. C'est ainsi qu'un microservice est lancé en production.

Dans la console, les dernières lignes doivent ressembler à :

```
... Tomcat started on port 8081 (http) with context path '/'
... Started CalculServiceApplication in 1.234 seconds
```

Le microservice tourne tant que la console est ouverte. Pour l'arrêter : bouton carré rouge dans IntelliJ, ou `Ctrl+C` dans le terminal. Après chaque modification du code, arrêtez-le et relancez-le (l'équivalent du « reconstruire, redéployer » du TP2).

Si le démarrage échoue avec `Port 8081 was already in use` : un autre programme occupe le port. Arrêtez-le, ou changez la valeur de `server.port`.

Lancer une seconde copie sur un autre port. Une propriété peut être remplacée au lancement, sans modifier `application.properties` : `java -jar target/calcul-service-0.0.1-SNAPSHOT.jar --server.port=8083`. L'option de la ligne de commande l'emporte sur le fichier. Vous en aurez besoin au TP4, où plusieurs exemplaires du même microservice tournent en même temps.

C.6 Comprendre l'URL du service

L'URL complète du service est :

```
http://localhost:8081/calcul/somme?a=2&b=3
```

Partie de l'URL	Origine
`localhost:8081`	Machine locale et port fixé par `server.port` dans `application.properties`
`/calcul`	Annotation `@RequestMapping("/calcul")` de la classe
`/somme`	***A compléter***
`?a=2&b=3`	***A compléter***

Question C.1. Comparez cette URL avec celle du TP2 (`http://localhost:8080/RestWebApp/api/calcul/somme?a=2&b=3`). Quelles parties ont disparu, et pourquoi ?

C.7 Tester le service

Avec le navigateur. Saisissez l'URL ci-dessus dans la barre d'adresse. Résultat attendu : la page affiche 5.

Avec curl. Dans un terminal, exécutez la commande suivante. Les guillemets autour de l'URL sont obligatoires, à cause du caractère &. L'option `-i` affiche le statut et les en-têtes de la réponse.

```
curl -i "http://localhost:8081/calcul/somme?a=2&b=3"
```

Résultat attendu : une première ligne `HTTP/1.1 200`, un en-tête `Content-Type: text/plain;charset=UTF-8`, puis le corps 5.

Avec Postman.

1. Créez une nouvelle requête (bouton **New** ou **+**).
2. Choisissez le verbe **GET** et saisissez l'URL.
3. Cliquez sur **Send**.
4. Observez le corps de la réponse (5), le statut (200 OK) et l'onglet **Headers**.

Question C.2. Remplacez GET par POST dans Postman et renvoyez la requête. Quel code de statut obtenez-vous ? Est-ce le même qu'au TP2 ?

Partie D : Compléter la calculatrice

Vous ajoutez les trois autres calculs à la classe `CalculController`, puis vous gérez un cas d'erreur.

D.1 Soustraction et produit (à faire seul)

En vous inspirant de la méthode `somme`, ajoutez dans `CalculController` :

- une méthode `soustraction` accessible par `GET /calcul/soustraction?a=..&b=..`
- une méthode `produit` accessible par `GET /calcul/produit?a=..&b=..`

Relancez le microservice, puis vérifiez :

Requête	Résultat attendu
`GET /calcul/soustraction?a=10&b=4`	6
`GET /calcul/produit?a=6&b=7`	42

D.2 Division et gestion d'erreur

Comme au TP2, diviser par zéro est une erreur du client : le service répond avec le statut 400 (*Bad Request*, requête incorrecte). Ajoutez cette méthode dans `CalculController` :

```
@GetMapping(value = "/division", produces = MediaType.TEXT_PLAIN_VALUE)
public ResponseEntity<String> division(@RequestParam("a") int a,
                                       @RequestParam("b") int b) {
    if (b == 0) {
        return ResponseEntity.badRequest()
                .body("Erreur : division par 0 impossible");
    }
    double result = calculService.division(a, b);
    return ResponseEntity.ok(String.valueOf(result));
}
```

`ResponseEntity.badRequest().body(...)` est l'équivalent de `Response.status(Response.Status.BAD_REQUEST).entity(...).build()` du TP2. Pour un autre statut, on écrit `ResponseEntity.status(HttpStatus.NOT_FOUND).build()` (import `org.springframework.http.HttpStatus`).

Relancez, puis testez avec `curl -i` :

Requête	Statut attendu	Corps attendu
`GET /calcul/division?a=7&b=2`	200	3.5
`GET /calcul/division?a=7&b=0`	400	Erreur : division par 0 impossible

D.3 Observations

Envoyez les deux requêtes suivantes et notez le statut et le corps de chaque réponse.

1. `GET /calcul/somme?a=2` (le paramètre b est absent)
2. `GET /calcul/somme?a=abc&b=3` (le paramètre a n'est pas un entier)

Question D.1. Au TP2, JAX-RS donnait la valeur 0 à un paramètre entier absent et la méthode `somme` s'exécutait. Que fait Spring dans le même cas ? Laquelle des deux attitudes vous paraît la plus conforme à REST ?

Question D.2. Avec curl ou Postman, Spring renvoie ses messages d'erreur sous forme d'objet JSON. Quels champs contient-il ?

D.4 Paramètres dans le chemin : @PathVariable

Les valeurs peuvent aussi être placées dans le chemin de l'URI, par exemple `/calcul/produit/6/7`. On les lit alors avec l'annotation `@PathVariable` (l'équivalent de `@PathParam` du TP2), qui nécessite l'import `org.springframework.web.bind.annotation.PathVariable`.

Ajoutez cette méthode dans `CalculController` :

```
@GetMapping(value = "/produit/{a}/{b}", produces = MediaType.TEXT_PLAIN_VALUE)
public ResponseEntity<String> produitChemin(@PathVariable("a") int a,
                                            @PathVariable("b") int b) {
    int result = calculService.produit(a, b);
    return ResponseEntity.ok(String.valueOf(result));
}
```

Test : `GET /calcul/produit/6/7` doit retourner 42.

Question D.3. Complétez le tableau de correspondance : à quelles annotations JAX-RS correspondent `@RequestParam` et `@PathVariable` ? Laquelle utilisera-t-on pour lire l'identifiant dans `/operations/{id}` ?

Partie E : Échanger des données en JSON

Jusqu'ici, le service renvoie un simple nombre en texte brut. Comme au TP2, vous lui faites maintenant renvoyer un objet structuré au format JSON. La conversion est assurée par **Jackson**, inclus dans `spring-boot-starter-webmvc` : il remplace JSON-B du TP2.

E.1 La classe Resultat

Créez la classe Créez la classe `Resultat` dans le paquetage `model`, identique à celle du TP2 à l'exception du paquetage. Jackson, comme JSON-B, a besoin d'un constructeur public sans argument et des accesseurs (méthodes get et set).

```
package com.exemple.calcul.model;

public class Resultat {

    private String operation;
    private int a;
    private int b;
    private double resultat;

    // Constructeur sans argument : obligatoire pour la conversion JSON
    public Resultat() {
    }
    public Resultat(String operation, int a, int b, double resultat) {
        this.operation = operation;
        this.a = a;
        this.b = b;
        this.resultat = resultat;
    }

    /* A compléter */
}
```

E.2 Une méthode qui produit du JSON

Ajoutez cette méthode dans `CalculController`. Avec `@RestController`, retourner un objet suffit : Spring le convertit en JSON et renvoie le statut 200. Il n'est même pas nécessaire d'écrire `produces`, car JSON est le format par défaut pour un objet. Pensez à ajouter dans `CalculController` l'import `com.exemple.calcul.model.Resultat`.

```
@GetMapping("/somme/detail")
public Resultat sommeDetail(@RequestParam("a") int a,
                            @RequestParam("b") int b) {
    return new Resultat("somme", a, b, calculService.somme(a, b));
}
```

Relancez, puis testez :

```
curl -i "http://localhost:8081/calcul/somme/detail?a=2&b=3"
```

Résultat attendu : le statut 200, l'en-tête `Content-Type: application/json` et le corps suivant (l'ordre des champs peut varier).

```
{"operation":"somme","a":2,"b":3,"resultat":5.0}
```

Question E.1. Au TP2, JSON-B avait produit `{"a":2,"b":3,"operation":"somme","resultat":5.0}`. Les deux textes JSON sont-ils équivalents pour un client ? Qui a effectué la conversion cette fois-ci ?

E.3 L'en-tête Accept

Le client indique le format qu'il souhaite recevoir avec l'en-tête HTTP `Accept`. Avec curl, l'option `-H` ajoute un en-tête à la requête. Envoyez ces deux requêtes et comparez les statuts :

```
curl -i -H "Accept: application/json"
"http://localhost:8081/calcul/somme/detail?a=2&b=3"
curl -i -H "Accept: application/xml"
"http://localhost:8081/calcul/somme/detail?a=2&b=3"
```

Résultat attendu : la première répond 200, la seconde répond 406 (*Not Acceptable*, non acceptable), exactement comme au TP2.

Question E.2. Pourquoi la seconde requête est-elle refusée, alors que le code de la méthode ne mentionne aucun format ?

E.4 Exercice

Ajoutez une méthode `divisionDetail`, accessible par `GET /calcul/division/detail?a=..&b=..`, qui renvoie un objet `Resultat` en JSON. Si b vaut 0, elle renvoie le statut 400. Indication : le type de retour `ResponseEntity<Resultat>` permet de choisir le statut tout en renvoyant un objet converti en JSON (`ResponseEntity.ok(resultat)` ou `ResponseEntity.badRequest().build()`).

Partie F : Second microservice : historique-service

Les calculs directs n'utilisent que le verbe GET. Pour pratiquer les quatre verbes, vous réalisez maintenant l'historique conçu à l'étape A.2, sous forme d'un **second microservice indépendant**. C'est ici que la différence avec le TP2 est la plus visible : au TP2, `OperationRest` appelait directement la méthode `calculService.somme(...)` ; ici, `historique-service` ne contient pas de classe `Calculatrice` et demande chaque résultat à `calcul-service` par une requête HTTP.

Les opérations sont conservées en mémoire, sans base de données. Elles sont donc perdues à chaque arrêt d'`historique-service`, et la numérotation repart alors de 1.

F.1 Créer le projet historique-service

Sur `https://start.spring.io`, générez un second projet exactement comme à l'étape B.1, avec les valeurs suivantes : **Artifact** `historique-service`, **Package name** `com.exemple.historique`, dépendance **Spring Web**. Ouvrez-le dans une seconde fenêtre IntelliJ (**File > Open**, puis **New Window**). Les deux projets doivent rester ouverts : vous lancerez les deux microservices en même temps. Créez ensuite sous `com.exemple.historique` les sous-paquetages `controller`, `service`, `client` et `model`, comme indiqué ci-dessous. Choisissez la même version de Spring Boot que pour `calcul-service` (4.0.9).

Dans `src/main/resources/application.properties`, fixez le port et l'adresse de `calcul-service` :

```
spring.application.name=historique-service
server.port=8082
calcul.service.url=http://localhost:8081
```

La troisième propriété n'est pas connue de Spring : c'est une propriété à vous, que vous lirez dans le code (étape F.3). Mettre cette adresse dans la configuration, et non dans le code, permet de déplacer `calcul-service` sur une autre machine sans recompiler.

Structure attendue à la fin de la partie F :

```
historique-service/
└── src/main/java/com/exemple/historique/
    ├── HistoriqueServiceApplication.java       (généré : démarrage)
    ├── controller/
    │   └── OperationController.java            (ressource /operations)
    ├── service/
    │   └── OperationService.java               (stockage en mémoire)
    ├── client/
    │   └── CalculClient.java                   (appel HTTP vers calcul-service)
    └── model/
        └── Operation.java                      (objet de l'historique)
```

F.2 La classe Operation

Une opération possède un identifiant, un opérateur, deux opérandes et un résultat. Le client fournit `operateur`, `a` et `b` ; le serveur calcule `id` et `resultat`. La classe est celle du TP2. Notez qu'elle est **recopiée** dans ce projet : chaque microservice possède son propre code et ne partage avec les autres que le contrat HTTP (URI, verbes, JSON). Placez-la dans le paquetage `model`.

```
package com.exemple.historique.model;

public class Operation {

    private int id;
    private String operateur; // somme, soustraction, produit ou division
    private int a;
    private int b;
    private double resultat;
```

```
    // Constructeur sans argument : obligatoire pour la conversion JSON
    public Operation() {
    }

    /* A Compléter */
}
```

F.3 L'appel HTTP vers calcul-service : classe CalculClient

Cette classe joue, dans `historique-service`, le rôle que jouait `Calculatrice` dans le TP2 : on lui demande un résultat. Mais au lieu de calculer, elle envoie une requête `GET /calcul/{operateur}?a=..&b=..` à `calcul-service` et lit la réponse. Elle utilise `RestClient`, le client HTTP de Spring. Placez-la dans le paquetage `client`.

```
package com.exemple.historique.client;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestClient;

@Service
public class CalculClient {

    private final RestClient client;

    // @Value lit la propriété calcul.service.url de application.properties
    public CalculClient(@Value("${calcul.service.url}") String url) {
        this.client = RestClient.create(url);
    }

    // Envoie GET {url}/calcul/{operateur}?a=..&b=.. et retourne le résultat.
    // En cas d'erreur (statut 4xx, service injoignable...), RestClient
    // lance une exception.
    public double calculer(String operateur, int a, int b) {
        String corps = client.get()
                .uri("/calcul/{op}?a={a}&b={b}", operateur, a, b)
                .retrieve()
                .body(String.class);
        return Double.parseDouble(corps);
    }
}
```

Points à comprendre :

- `RestClient.create(url)` construit un client dont toutes les URI commenceront par `url`, c'est-à-dire `http://localhost:8081`.
- `uri("/calcul/{op}?a={a}&b={b}", operateur, a, b)` remplace les variables `{op}`, `{a}` et `{b}` par les valeurs fournies, dans l'ordre.
- `retrieve().body(String.class)` envoie la requête et retourne le corps de la réponse sous forme de texte. `calcul-service` répond en `text/plain` (« 5 » ou « 3.5 ») ; `Double.parseDouble` convertit ce texte en nombre.

- Si `calcul-service` répond 400 (division par 0), `RestClient` lance une `HttpClientErrorException`. S'il est injoignable (arrêté), il lance une `ResourceAccessException`. Le contrôleur les traitera à l'étape F.5.

F.4 Le stockage : classe OperationService

Cette classe conserve les opérations en mémoire et attribue les identifiants. Placez-la dans le paquetage `service`.

```
package com.exemple.historique.service;

import com.exemple.historique.model.Operation;

import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;

@Service
public class OperationService {

    // Pas de "static" : Spring ne crée qu'un seul OperationService
    // (voir la remarque ci-dessous)
    private final Map<Integer, Operation> operations =
            new ConcurrentHashMap<>();
    private final AtomicInteger compteur = new AtomicInteger(0);

    public Operation ajouter(Operation op) {
         /* A compléter */
    }

    public List<Operation> lister() {
        /* A compléter */
    }

    public Operation lire(int id) {
        /* A compléter */      // null si l'identifiant n'existe pas
    }

    public boolean existe(int id) {
        /* A compléter */
    }

    public void remplacer(int id, Operation op) {
        /* A compléter */
    }

    public Operation supprimer(int id) {
        /* A compléter */   // null si l'identifiant n'existait pas
    }
}
```

Remarque : plus de static. Au TP2, les attributs `OPERATIONS` et `COMPTEUR` devaient être `static`, parce que JAX-RS créait un nouvel objet `OperationRest` à chaque requête. Avec Spring, un objet annoté `@Service` est un *singleton* : il est créé une seule fois au démarrage et partagé par toutes les requêtes. Des attributs ordinaires suffisent donc. Comme plusieurs requêtes peuvent arriver en même temps, on garde des structures prévues pour les accès simultanés : `ConcurrentHashMap` et `AtomicInteger`.

F.5 Le contrôleur : classe OperationController (créer et lire)

Créez la classe `OperationController` avec le code suivant. Il réalise la création (POST) et les deux lectures (GET). Le contrôleur reçoit ses deux dépendances par le constructeur. Placez-le dans le paquetage `controller`.

```
package com.exemple.historique.controller;

import com.exemple.historique.client.CalculClient;
import com.exemple.historique.model.Operation;
import com.exemple.historique.service.OperationService;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.ResourceAccessException;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

import java.net.URI;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/operations")
public class OperationController {

    private static final List<String> OPERATEURS =
            List.of("somme", "soustraction", "produit", "division");

    private final OperationService operationService;
    private final CalculClient calculClient;

    public OperationController(OperationService operationService,
                               CalculClient calculClient) {
        /* A compléter */
    }

    // Demande le résultat à calcul-service et le range dans l'opération.
    // Retourne une réponse d'erreur si le calcul est impossible, null sinon.
    private ResponseEntity<Object> calculer(Operation op) {
       /* A compléter */
    }
```

```
    // POST /operations : crée une opération
    @PostMapping
    public ResponseEntity<Object> creer(@RequestBody Operation op) {
        /* A compléter */
    }

    // GET /operations : retourne toutes les opérations
    @GetMapping
    public List<Operation> lister() {
        /* A compléter */
    }

    // GET /operations/{id} : retourne une opération
    @GetMapping("/{id}")
    public ResponseEntity<Operation> lire(@PathVariable("id") int id) {
        /* A compléter */
    }
}
```

Points à comprendre :

- `@RequestBody Operation op` : Spring lit le corps JSON de la requête et le convertit en objet `Operation`. C'est l'équivalent du paramètre `Operation op` accompagné de `@Consumes(MediaType.APPLICATION_JSON)` au TP2.
- `ResponseEntity<Object>` permet de renvoyer tantôt une `Operation`, tantôt un message d'erreur. `Map.of("erreur", "...")` est converti par Jackson en `{"erreur":"..."}`, sans écrire le JSON à la main comme au TP2.
- `ServletUriComponentsBuilder.fromCurrentRequest().path("/{id}")` construit l'URI de la nouvelle ressource à partir de celle de la requête ; c'est l'équivalent de `uriInfo.getAbsolutePathBuilder()`. `ResponseEntity.created(location)` renvoie le statut 201 (*Created*) avec l'en-tête `Location`.
- `ResponseEntity.notFound().build()` renvoie le statut 404 (*Not Found*) sans corps.
- Le statut 503 (*Service Unavailable*, service indisponible) indique au client que le serveur ne peut pas répondre pour l'instant ; c'est la réponse appropriée quand un microservice dont on dépend est arrêté.

F.6 Tester la création et la lecture

Lancez `historique-service` (comme à l'étape C.5), en gardant `calcul-service` en marche : vous devez avoir deux consoles, l'une indiquant le port 8081 et l'autre le port 8082. Envoyez ensuite les requêtes suivantes **dans l'ordre**.

Création avec curl, sous Linux, macOS ou Git Bash :

```
curl -i -X POST -H "Content-Type: application/json" -d
'{"operateur":"somme","a":8,"b":2}' "http://localhost:8082/operations"
```

Dans l'invite de commandes Windows (cmd), les guillemets internes doivent être précédés d'une barre oblique inverse :

```
curl -i -X POST -H "Content-Type: application/json" -d
"{\"operateur\":\"somme\",\"a\":8,\"b\":2}" "http://localhost:8082/operations"
```

Création avec Postman :

1. Choisissez le verbe **POST** et l'URL `http://localhost:8082/operations`.
2. Ouvrez l'onglet **Body**, sélectionnez **raw**, puis le type **JSON**.
3. Saisissez le corps `{"operateur":"somme","a":8,"b":2}` et cliquez sur **Send**.

Avant ces tests, vous pouvez ajouter une trace (par exemple un `System.out.println` dans la méthode `somme` de `CalculController`) et relancer `calcul-service` : à chaque POST, la trace s'affiche dans sa console, ce qui montre que la requête passe bien par lui.

Série de tests :

N°	Requête	Corps envoyé	Statut	Réponse attendue
1	`POST /operations`	`{"operateur":"somme","a":8,"b":2}`	/* A compléter */	/* A compléter */
2	`POST /operations`	`{"operateur":"division","a":7,"b":2}`	/* A compléter */	/* A compléter */
3	`GET /operations`	Aucun	/* A compléter */	/* A compléter */
4	`GET /operations/2`	Aucun	/* A compléter */	/* A compléter */
5	`GET /operations/99`	Aucun	/* A compléter */	/* A compléter */
6	`POST /operations`	`{"operateur":"division","a":7,"b":0}`	/* A compléter */	/* A compléter */
7	`POST /operations`	`{"operateur":"racine","a":9,"b":0}`	/* A compléter */	/* A compléter */

Toutes les URI de ce tableau sont à préfixer par `http://localhost:8082`. L'ordre des champs dans le JSON peut varier.

F.7 Expérience : arrêter un microservice

Cette expérience met en évidence l'indépendance des deux microservices. **Arrêtez `calcul-service`** (bouton carré rouge de sa console) et laissez `historique-service` en marche. Envoyez ensuite :

N°	Requête	Corps envoyé	Statut	Réponse attendue
8	`GET /operations`	Aucun	/* A compléter */	/* A compléter */
9	`GET /operations/1`	Aucun	/* A compléter */	/* A compléter */
10	`POST /operations`	`{"operateur":"produit","a":3,"b":4}`	/* A compléter */	/* A compléter */

Relancez `calcul-service`, puis renvoyez la requête 10 : elle doit maintenant répondre 201 avec l'identifiant 3.

Question F.1. Pourquoi les lectures continuent-elles de fonctionner alors que `calcul-service` est arrêté ? Une telle situation était-elle possible au TP2 ?

Question F.2. Pendant l'arrêt de `calcul-service`, le client a reçu une réponse 503 en moins d'une seconde. Que se serait-il passé sans le bloc `catch (ResourceAccessException e)` ? Testez-le en le mettant en commentaire, puis remettez-le.

F.8 Modifier une opération : PUT (à faire seul)

Ajoutez dans `OperationController` une méthode `modifier` qui répond à `PUT /operations/{id}`. Elle reçoit en JSON une opération (`operateur`, `a`, `b`) qui remplace celle d'identifiant `id`.

Comportement attendu :

Situation	Statut	Corps de la réponse
Aucune opération ne porte cet identifiant	/* A compléter */	/* A compléter */
Le calcul est impossible (opérateur inconnu, division par 0)	/* A compléter */	/* A compléter */
calcul-service est injoignable	/* A compléter */	/* A compléter */
Modification réussie	/* A compléter */	/* A compléter */

Indications :

- annotation nécessaire : `@PutMapping("/{id}")` (import `org.springframework.web.bind.annotation.PutMapping`)
- signature : `public ResponseEntity<Object> modifier(@PathVariable("id") int id, @RequestBody Operation op)`
- vérifiez d'abord `operationService.existe(id)`, puis réutilisez la méthode `calculer`, puis `operationService.remplacer(id, op)`.

Test : `PUT /operations/1` avec le corps `{"operateur":"produit","a":8,"b":2}` doit répondre 200 avec un résultat de 16.0. Vérifiez ensuite avec `GET /operations/1`.

F.9 Supprimer une opération : DELETE (à faire seul)

Ajoutez une méthode `supprimer` qui répond à `DELETE /operations/{id}`.

Situation	Statut	Corps de la réponse
Aucune opération ne porte cet identifiant	/* A compléter */	/* A compléter */
Suppression réussie	/* A compléter */	/* A compléter */

Indications :

- annotation nécessaire : `@DeleteMapping("/{id}")` (import `org.springframework.web.bind.annotation.DeleteMapping`)
- `operationService.supprimer(id)` retourne `null` si l'opération n'existait pas
- `ResponseEntity.noContent().build()` construit une réponse de statut 204.

Tests, dans l'ordre :

```
curl -i -X DELETE "http://localhost:8082/operations/1"
curl -i "http://localhost:8082/operations/1"
curl -i -X DELETE "http://localhost:8082/operations/1"
```

Résultats attendus : 204, puis 404, puis 404.

Questions de synthèse

1. Reprenez le tableau A.3 et, pour chaque ligne, indiquez l'étape du TP où vous avez observé la différence.
2. Dressez le tableau de correspondance complet entre les annotations JAX-RS utilisées au TP2 et les annotations Spring utilisées dans ce TP.
3. Au TP2, une seule commande (`mvn clean package`) produisait un fichier .war qu'il fallait ensuite déployer dans Tomcat. Décrivez les commandes nécessaires pour construire et lancer les deux microservices de ce TP sans IntelliJ.
4. Un microservice est-il un service web ? Un service web est-il toujours un microservice ? Justifiez à l'aide de ce TP.
5. Dans ce TP, l'adresse de `calcul-service` est écrite dans `application.properties` d'`historique-service`. Quel problème cela pose-t-il si l'on lance trois exemplaires de `calcul-service` pour répartir la charge ? (Ce problème sera traité au TP4.)
6. Le principe *stateless* (sans état) interdit au serveur de mémoriser l'état du client entre deux requêtes. `historique-service` conserve pourtant la liste des opérations. Expliquez pourquoi cela ne contredit pas ce principe.
7. Vous envoyez deux fois la même requête `POST /operations`, puis deux fois la même requête `PUT /operations/1`. Comparez l'effet de la répétition dans les deux cas. La réponse est-elle différente de celle du TP2 ?

Travail à rendre dans le dépôt Git et le fichier README

- Les deux projets Maven complets (`calcul-service` et `historique-service`), chacun dans son dossier.
- Les réponses aux questions des parties A, C, D, E et F et aux questions de synthèse.
- Pour chaque test des étapes F.6, F.7, F.8 et F.9 : la requête envoyée, le statut et le corps obtenus (capture d'écran Postman ou sortie de `curl -i`).
- Une capture d'écran montrant les deux consoles avec les lignes `Tomcat started on port 8081` et `Tomcat started on port 8082`.